Below is a **master table of the most important and detailed Microservices terminology** used across *all generations/versions of microservices architecture* (from early SOA → Cloud Native → Kubernetes/Service Mesh → Event-driven → Serverless → Platform Engineering), along with **real-time use cases + examples**.

✅ Microservices Terminology (Master Table) + Real-Time Use Cases + Examples

Terminology	Meaning (What it is)	Real-time Use Case	Example (Software / Practical)
Microservice	Small independently deployable service	Separate teams deploy faster	payment-service, order-service
Monolith	Single deployable app with all modules	Good for early stage / MVP	One Spring Boot app handling all
Modular Monolith	Monolith but well-separated modules	Ideal before splitting to microservices	billing-module, user-module inside one codebase
SOA (Service Oriented Architecture)	Earlier services architecture	Used in enterprises before microservices	Banking SOAP based services
Bounded Context	DDD boundary where model is consistent	Avoids domain confusion	Order bounded context ≠ Inventory
Domain Driven Design (DDD)	Design based on business domain	Complex business systems	Banking, Insurance systems
Ubiquitous Language	Common business + tech vocabulary	Avoid mismatch in requirements	"Policy", "Claim", "Premium" terms
Aggregate	Cluster of domain objects as a unit	Ensures consistency rules	Order aggregate containing items
Entity	Object with identity	Persistent business objects	Customer entity with id
Value Object	Object without identity	Represents immutable values	Money (amount, currency)
API Gateway	Entry point for external clients	Authentication, routing, rate limits	Netflix Zuul / Spring Cloud Gateway
Reverse Proxy	Routes traffic to internal services	Load balancing and security	NGINX, Envoy

Load Balancer	Distributes traffic across instances	Scaling services	AWS ELB / NGINX LB
Service Registry / Discovery	Services discover each other dynamically	No hard-coded IPs	Eureka / Consul
Client-side Discovery	Client finds service from registry	Used with Eureka	Feign client picks instance
Server-side Discovery	Proxy finds service	Kubernetes default discovery	Ingress routes to pods
Sidecar Pattern	Helper container for service features	Used in service mesh	Envoy sidecar in Istio
Service Mesh	Network layer for microservices	mTLS, traffic control	Istio / Linkerd
mTLS	Mutual TLS between services	Zero trust security	Service-to-service encryption
Circuit Breaker	Stops calling failing service temporarily	Prevent cascade failures	Resilience4j circuit breaker
Bulkhead Pattern	Isolate resources per service	Prevent one service taking all threads	Separate thread pool for Payment
Retry Pattern	Retry failed calls	Network timeouts	Retry 3 times with backoff
Timeout	Max wait before failing request	Prevent stuck threads	HTTP client timeout 2s
Rate Limiting	Restrict number of requests	Protect services from abuse	100 req/min per user
Throttling	Slow down excessive requests	Avoid overload	Return 429 Too Many Requests
API Versioning	Manage multiple versions	Backward compatibility	/api/v1/users, /api/v2/users
Backward Compatibility	Older clients still work	Mobile apps update slowly	Keep v1 fields alive
Contract Testing	Validate service interface	Prevent breaking changes	Pact tests

Consumer Driven Contract (CDC)	Consumer defines contract	API changes controlled	Mobile app defines expected response
Synchronous Communication	Request-response (HTTP/gRPC)	Real-time operations	Payment check at checkout
Asynchronous Communication	Message-based	High throughput and decoupling	Order placed → event to inventory
Event Driven Architecture (EDA)	Services react to events	Massive scalability	Uber / Amazon style architecture
Event	Something happened	Data propagation	OrderCreated event
Event Stream	Continuous flow of events	Real-time analytics	Kafka topic orders
Broker	Message/event transport	Async communication backbone	Kafka / RabbitMQ
Topic	Kafka stream channel	Separate events per domain	payment-events
Queue	FIFO message container	Worker processing	email-notification-queue
Pub/Sub	Many consumers listen to an event	Notify multiple services	Order event to Email + Analytics
Saga Pattern	Distributed transactions workflow	Data consistency across services	Order→Payment→Inventory saga
Choreography Saga	No central controller	Simple sagas	Services react to events
Orchestration Saga	Central controller manages flow	Complex workflows	Camunda / Temporal orchestration
Two-Phase Commit (2PC)	Distributed ACID commit	Rare in microservices	Avoided due to locking
Eventually Consistent	Data becomes consistent later	High scale architecture	Inventory updates after order
Idempotency	Repeating request doesn't duplicate	Payment safety	Same paymentId processed once

Correlation ID	Unique ID per request journey	Tracing	X-Correlation-ID header
Distributed Tracing	Track request across services	Debug latency issues	Jaeger / Zipkin
Observability	Logs + metrics + traces	Production monitoring	OpenTelemetry
Structured Logging	JSON logs with fields	Better debugging	{service:"order",level:"INFO"}
Metrics	Numeric health data	Scaling decisions	CPU, latency p95
Golden Signals	Latency, Traffic, Errors, Saturation	SRE monitoring basics	Grafana dashboards
SLA	Service Level Agreement	Business commitment	“99.9% uptime”
SLO	Target reliability	Internal target	99.95% service uptime
SLIs	Actual reliability metrics	Measure SLO	Error rate, p99 latency
Blue-Green Deployment	Switch between two envs	Zero downtime release	Blue running, Green new
Canary Deployment	Release to small % first	Reduce blast radius	5% traffic to new Payment
Rolling Update	Replace pods gradually	Default k8s upgrade	1 pod at a time update
Feature Flag	Toggle features dynamically	Safe releases	Launch new UI for 10% users
CI/CD	Automated build-test-deploy	Faster releases	Jenkins/GitHub Actions pipelines
GitOps	Git is source of truth for deployments	Stable deployments	ArgoCD applies K8s manifests
Kubernetes (K8s)	Container orchestration	Manage microservices infra	Pods, deployments, services
Pod	Smallest deployable unit	Runs containers	order-service pod
Deployment	Controller for pods	Scaling + rollout	replicas=5

ReplicaSet	Ensures #pods running	Stability	Always keep 3 pods
HPA	Horizontal Pod Autoscaler	Auto scale based on CPU/RPS	scale 3 → 15 pods
Ingress	Entry routing to cluster	Public access	api.company.com routes
ConfigMap	External config	Environment-specific configs	DB_HOST, timeout
Secret	Store sensitive info	Keys, passwords	JWT secret, DB password
Helm	Kubernetes package manager	Deploy templates easily	helm install payments
Service (K8s)	Stable networking endpoint	Service-to-service communication	ClusterIP service for user-service
Namespace	Logical partition in cluster	Multi-team isolation	dev, staging, prod
Docker Container	Runtime packaging	Portability	order-service container image
Image Registry	Store images	Deployment source	DockerHub / ECR
Stateless Service	No session stored locally	Easy scaling	APIs with Redis session
Stateful Service	Requires persistent state	Databases	PostgreSQL, MongoDB
Database per Service	Each service owns its DB	Avoid coupling	payment-db, order-db
Shared Database	Multiple services use same DB	Bad practice (usually)	Order + Payment using same DB
Polyglot Persistence	Different DB per service	Optimal storage models	Mongo for catalog, SQL for orders
CQRS	Separate read and write models	Heavy read systems	Dashboard reads from projection DB
Command	Write operation	Change state	CreateOrderCommand

Query	Read operation	Fetch data	GetOrderStatusQuery
Read Model	Optimized for read	Fast UI response	ElasticSearch projection
Event Sourcing	State derived from events	Audit + rebuild	Order events rebuild order state
Anti-corruption Layer (ACL)	Protect domain from external models	Safe integration	Legacy ERP adapter
Strangler Fig Pattern	Gradual monolith replacement	Safe migration	Build new services around monolith
API Composition	Aggregating data from many services	Reduce client calls	BFF merges order+customer
Backend for Frontend (BFF)	API tailored per client type	Mobile vs Web differences	mobile-bff, web-bff
Data Mesh	Domain teams own data products	Large organizations	Finance data product owned by finance team
Platform Engineering	Internal platform (IDP) for teams	Developer productivity	Self-service deploy portal
IDP (Internal Dev Platform)	Dev self-service platform	Standardization	Backstage portal
12-Factor App	Cloud-native app principles	Portable services	config via env vars
Zero Trust	Never trust network by default	Secure microservices	mTLS + auth everywhere
OAuth2	Authorization framework	Secure APIs	Google login
OpenID Connect	Authentication layer on OAuth2	SSO	Login token identity
JWT	Token-based identity	Stateless auth	Authorization: Bearer <jwt>
RBAC	Role based access control	Admin vs user access	Admin can delete users
ABAC	Attribute based access control	Fine-grained permissions	Allow access if dept=HR

WAF	Web Application Firewall	Block attacks	Cloudflare WAF
Chaos Engineering	Inject failures intentionally	Improve resilience	Kill pods randomly
Fault Injection	Simulate latency/errors	Stability tests	Add 500ms delay in payment
Latency Budget	Max allowed latency	Maintain UX	Checkout must be < 2s
Hotspot Service	Service receiving heavy load	Optimize scaling	catalog-service during sale
Blast Radius	Scope of failure impact	Reduce outage	Canary reduces blast radius
Multi-tenancy	Multiple customers share infra	SaaS architecture	tenantId in requests

---

✅ Microservices Terminology by "Versions / Evolution"

(so you can explain in interviews or in architecture design)

Microservices Version	Key Focus	Technologies
v0 (SOA era)	SOAP, ESB, centralized integration	ESB, WSDL, SOAP
v1 (Microservices baseline)	REST, independent deployability	Spring Boot, REST, Docker
v2 (Cloud-native)	Containers, orchestration	Kubernetes, Helm
v3 (Resilience + Observability)	patterns + SRE	Circuit breaker, OTel, Prometheus
v4 (Event-driven Microservices)	async + scalable pipelines	Kafka, RabbitMQ, Saga
v5 (Service Mesh + Zero Trust)	mTLS + traffic shaping	Istio, Envoy, Linkerd
v6 (Serverless + Managed services)	no infra mgmt	AWS Lambda, API Gateway
v7 (Platform Engineering / IDP)	Developer productivity	Backstage, GitOps, ArgoCD
v8 (Data Mesh + AI-driven ops)	decentralize data +	Data mesh, anomaly detection

	AIOps	

If you want, I can also create **a complete microservices project sample** with:
✅ Spring Boot microservices (User + Order + Payment)
✅ API Gateway + Eureka
✅ Kafka events + Saga
✅ Docker Compose / Kubernetes YAML
✅ Observability (Prometheus + Grafana + Jaeger)

…and explain each term inside the code (real production style).